

COMP2017 Practice Exam B

Final-style paper: MCQ, memory snapshot, large pointer simulation, anonymous-pipe routing

Question	Topic	Marks
Q1-Q4	Multiple choice fundamentals	5
Q5	Memory snapshot	15
Q6	Large programming question	50
Q7	Processes and anonymous-pipe routing	30
Total		100

Instructions

- Answer all questions. Write clear C or pseudocode where allowed.
- Unless a question asks for defensive programming, assume inputs and system calls are valid.
- Where synchronization is required, avoid data races and busy waiting.
- State reasonable assumptions when the question leaves details unspecified.

Questions 1-4 - Multiple Choice (5 marks)

Question 1. What is the size of a **double *** type on a machine with 64-bit addressable memory?

- A. 4 bytes B. 8 bytes C. sizeof(double) D. depends on the value stored in the pointer

Question 2. What is the expected output of this program?

```
void f(int x) {
    static int y = 2;
    if (x % 2 == 0) {
        printf("%d ", x + y);
        y++;
    }
}

int main(void) {
    for (int i = 0; i < 5; i++) f(i);
}
```

- A. 2 5 8 B. 2 4 6 C. 2 6 D. segmentation fault

Question 3. Which statement best describes a semaphore?

- A. A condition variable that stores strings. B. A counting synchronization object. C. A replacement for fork. D. A pointer to a thread stack.

Question 4. Multiple answer, worth 2 marks. Which of the following are necessary conditions for deadlock?

- Mutual exclusion
- Hold and wait
- No preemption
- Circular wait
- Memory leak
- Buffer overflow
- Compiler warnings
- No file descriptors open

Question 5 - Memory Snapshot (15 marks)

Write the memory contents after execution of all statements up to the marked line. Write variable names, addresses and associated data. Starting segment addresses are provided. Assume memory is allocated positively from low addresses to high addresses. sizeof(char*) is 4 bytes. Assume no word alignment.

```
static char flag = 12;

int main(int argc, char **argv) {
    char *word = "CAB";
    char c = word[1];
    char *fp = &flag;
    char x = *fp + c;
    char **box = malloc(sizeof(char *));
    *box = fp;
    c = **box + 3;
    ; // show snapshot here
}
```

Program code starts at 0x2000
Static/global starts at 0x3000
Heap starts at 0x4000
Stack starts at 0x7000
ASCII 'A' = 65

Indicate where the string literal is stored, where local variables are stored, what value is stored in the heap object, and the final values of **c** and **x**.

Question 6 - Programming / Pointer Simulation (50 marks)

You are to complete part of a simulation for **n maintenance robots** moving along a one-dimensional rail. Each robot has a position, a velocity, and a reference to the robot in front of it. Robots may change references when they overtake or separate.

```
typedef int (*beep_f)(int);

struct robot {
    float position[1];
```

```

float velocity[1];
struct robot *robot_in_front;
beep_f beep;
};

extern beep_f calm_beep, warning_beep, error_beep, neutral_beep;
extern int same_direction(struct robot *, struct robot *);
extern void update_robots(struct robot **robots, int n, int max,
                        float timestep, int *removed);

```

Part 1 - 5 marks. Write **same_direction**. It returns >0 if both robots move in the same direction, 0 if either velocity is zero, and <0 otherwise. Behaviour is undefined if either argument is NULL.

```
int same_direction(struct robot *r1, struct robot *r2);
```

For Parts 2-5, write the complete function below. Each part must be performed in order. The array **robots** has size **max** and contains **n** active robots. Some elements may be NULL and should be skipped. Non-NULL robots are heap allocated. You cannot duplicate O(n) memory.

```
void update_robots(struct robot **robots, int n, int max,
                  float timestep, int *removed);
```

Part 2 - 13 marks. Update each robot position using $p' = p + v * \text{timestep}$. A robot cannot occupy a position within radius 1.5 of any other robot. If its proposed new position is too close, it should iteratively try $p' + v * k$ for integer $k > 0$ until it finds a valid position.

Part 3 - 13 marks. Overtaking occurs when the current robot and the robot in front move in the same direction and the current robot's position after Part 2 is further than the robot in front in that direction. Update the two **robot_in_front** references so that the current robot overtakes. Set current robot beep to **error_beep** and the overtaken robot beep to **warning_beep**. Repeat until no more overtakes occur. State how you revisit robots.

Part 4 - 6 marks. If a current robot and its robot_in_front do not move in the same direction, set current robot's **robot_in_front** to NULL and set the former front robot's beep to **warning_beep**.

Part 5 - 13 marks. A robot is removed when $\text{fabs}(\text{velocity}[0]) < 0.01$. Free its memory and set its array slot to NULL. If any robot has **warning_beep** and refers to a robot that has been removed, it is also removed. Repeat until stable. Any still-alive robot referring to a removed robot should have that reference changed to NULL. Initially ***removed** should be zero and incremented for each freed robot. If no robot was removed, all remaining robots use **calm_beep**; otherwise all remaining robots use **neutral_beep**.

Useful function: `double fabs(double x)`. You do not need to maintain the order of the array.

Question 7 - Processes / Anonymous-Pipe Routing (30 marks)

A program creates n processes $R_0, R_1, \dots, R_{n-1}$ in a single chain. R_0 is the original process. R_i is the parent of R_{i+1} . The hierarchy cannot be changed. The program must transfer one message from any process to a specific destination process using only anonymous pipes.

You may define any helper structures. Each transmitted message must include a fixed payload field **char msg[50]**.

Your answer must include:

- 8 marks: Explain how pipes are created for all processes in the chain. Write `create_children(...)` and state which process calls it.
- 10 marks: Define the message encoding and decoding protocol. Write `send_message(int dst, char *msg, int msg_len)` and `receive_message(char *buffer, int *nread, int buffer_max)`.
- 4 marks: Write `free_resources(...)` called after SIGTERM.
- 4 marks: Explain why your routing prevents cycles. For example, if R_7 sends to R_2 , R_7 must not receive its own message.
- 4 marks: Explain why the message converges and the destination receives it exactly once.

Functions may be written in C or clear pseudocode. You must use anonymous pipes only; do not use sockets, named pipes, files, or shared memory.

Examiner Blueprint

This page is not a solution guide. It records the intended assessment focus behind the paper.

Question	Intended signal	Core skills tested
Q1-Q4	Small fundamentals	Pointer size, static local lifetime, semaphore/mutex concepts, Coffman deadlock conditions.
Q5	Memory model	Literal vs static vs heap vs stack, pointer-to-pointer, char arithmetic, address/value separation.
Q6	Large programming	Struct pointer graph mutation, repeated passes to stable state, heap free, NULL slots, no $O(n)$ duplicate array.
Q7	IPC design	Anonymous pipe topology, fixed message struct, routing direction, cleanup on SIGTERM, proof of no cycles and convergence.